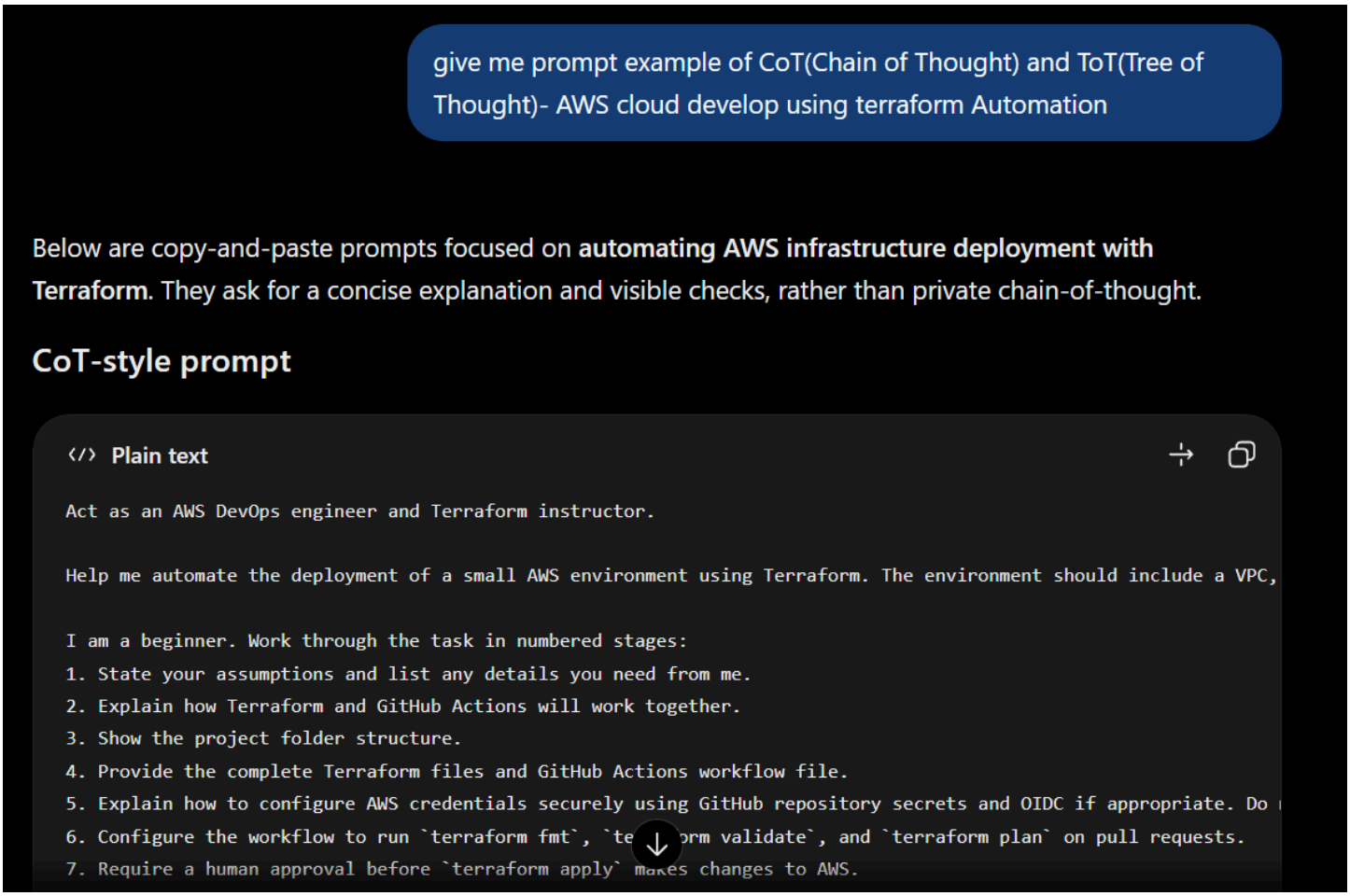
CoT- Style Prompt



Output
Act as an AWS DevOps engineer and Terraform instructor.

Help me automate the deployment of a small AWS environment using Terraform. The environment should include a VPC, two public subnets, an internet gateway, route tables, and one Amazon Linux EC2 instance. Use GitHub Actions to run Terraform automatically when I push changes to the main branch.

I am a beginner. Work through the task in numbered stages:

1. State your assumptions and list any details you need from me.
2. Explain how Terraform and GitHub Actions will work together.
3. Show the project folder structure.
4. Provide the complete Terraform files and GitHub Actions workflow file.
5. Explain how to configure AWS credentials securely using GitHub repository secrets and OIDC if appropriate. Do not put credentials in code.
6. Configure the workflow to run `terraform fmt`, `terraform validate`, and `terraform plan` on pull requests.
7. Require a human approval before `terraform apply` makes changes to AWS.

8. Give setup instructions and commands to test the workflow.
9. Explain how to verify and destroy the resources.

Use variables for configurable values. Restrict SSH access to an IP address supplied as a variable. Before presenting the files, check that names and references match across the Terraform configuration and workflow.

Show the final files and a concise explanation of the design. Mention cost factors and any assumptions. Do not claim the workflow was tested unless you actually tested it.

ToT-style prompt

Act as an AWS DevOps architect who teaches Terraform automation to beginners.

I want to automate AWS infrastructure deployment with Terraform. Compare these three automation approaches:

A. Run Terraform locally from my computer

B. Use GitHub Actions with AWS credentials stored as repository secrets

C. Use GitHub Actions with AWS IAM OIDC federation

For each approach, summarise:

- How the automation works
- Setup complexity
- Credential security
- Cost considerations
- Best use case

Compare them for a learning project that should be secure, low-cost, and easy to maintain. Recommend one approach and briefly explain why. Do not provide private step-by-step reasoning; show only the comparison and concise justification.

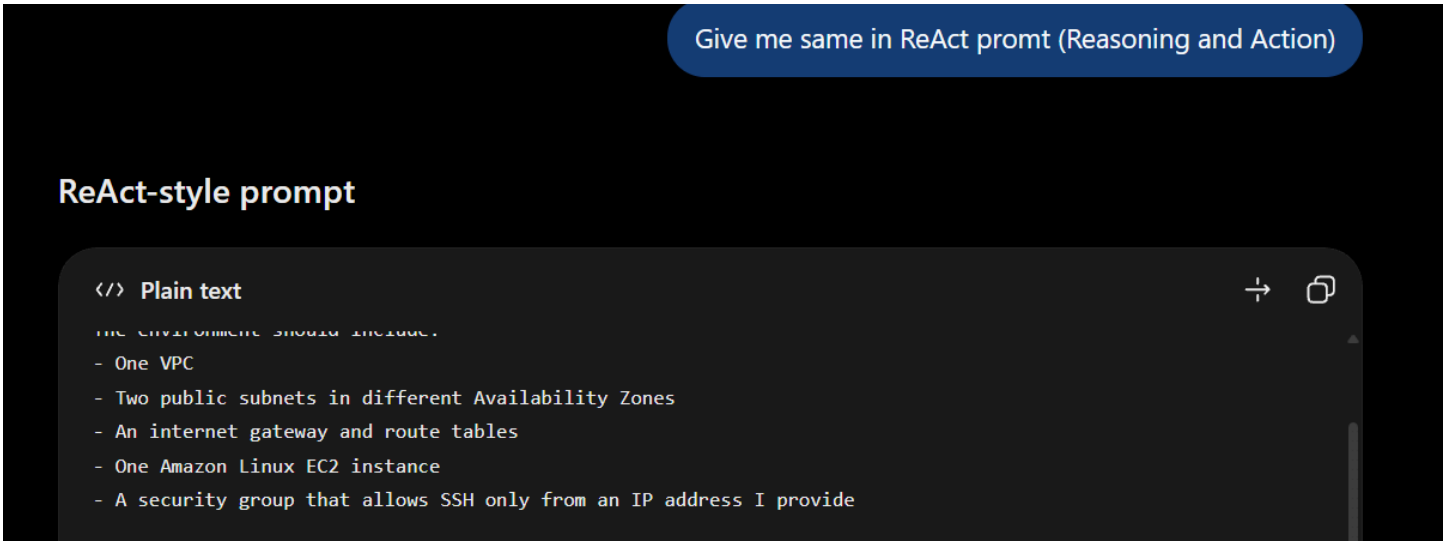
Then create a working starter project for the recommended approach:

- Terraform configuration for a VPC, two subnets, an internet gateway, route tables, and one EC2 instance
- A GitHub Actions workflow that runs `fmt`, `validate`, and `plan` on pull requests
- A separate approved step for `apply` after changes are merged
- Secure AWS authentication without credentials hard-coded in files
- Variables and outputs for configurable values and useful results

- A README with setup, testing, verification, and destroy instructions

Show the folder structure and the full contents of each file. Check that the Terraform references, workflow paths, and variable names are consistent. Include the permission and approval settings needed for the workflow, and point out any values I must configure before running it. Do not claim you tested the project unless you actually tested it.

ReAct-style prompt



Act as an AWS DevOps engineer and Terraform instructor. Help me automate deployment of a small AWS environment with Terraform and GitHub Actions.

The environment should include:

- One VPC
- Two public subnets in different Availability Zones
- An internet gateway and route tables
- One Amazon Linux EC2 instance
- A security group that allows SSH only from an IP address I provide

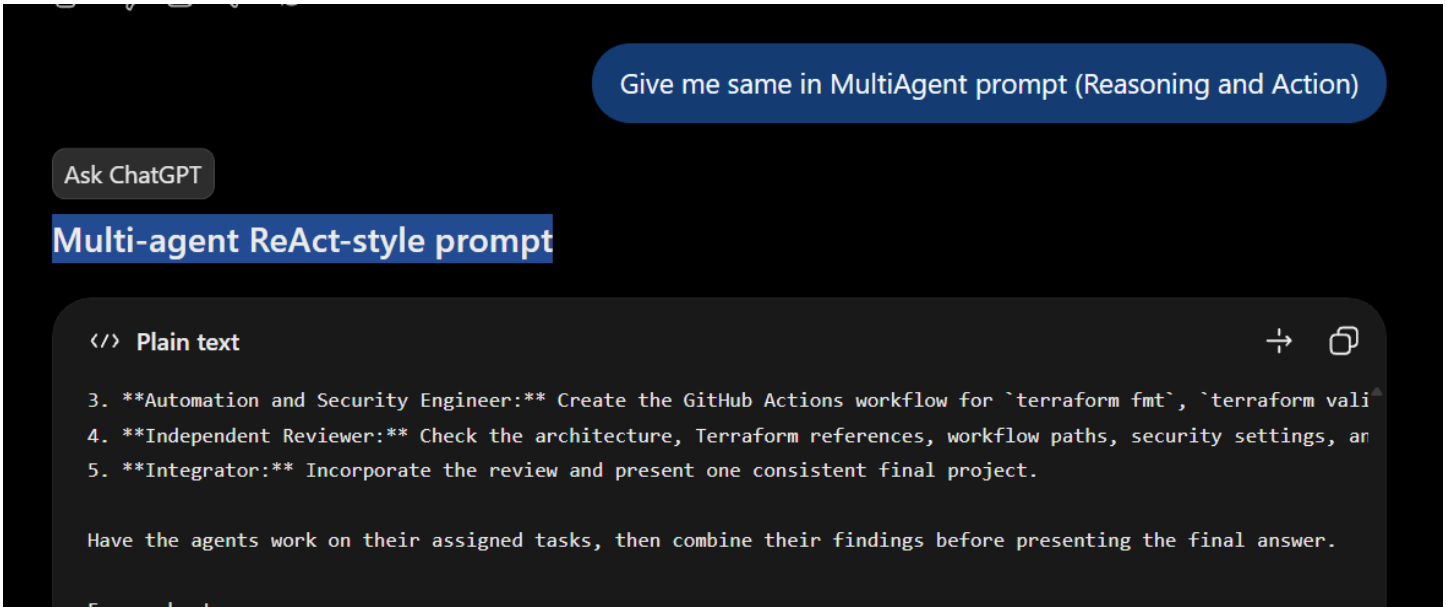
I am a beginner. Use this Reason–Action–Observation–Refine process. Show concise explanations and results at each stage; do not reveal private chain-of-thought.

1. **Reason:** List your assumptions and identify information you need from me. If something is missing, choose a safe example value and clearly mark it for me to replace.
2. **Action:** Explain the architecture and create a project folder structure.
3. **Observation:** Check that the structure meets the requirements and identify any missing configuration.

4. **Refine:** Provide the complete Terraform files, using variables for configurable values and outputs for useful results.
5. **Action:** Create a GitHub Actions workflow that runs `terraform fmt`, `terraform validate`, and `terraform plan` on pull requests.
6. **Observation:** Check that the workflow points to the correct Terraform directory and that its variable names and resource references match.
7. **Refine:** Add a deployment step that requires human approval before `terraform apply` changes AWS resources. Use secure AWS authentication, preferably GitHub Actions OIDC. Do not hard-code AWS credentials.
8. **Action:** Give beginner-friendly setup, testing, verification, and destroy instructions.
9. **Observation:** Review the final files for consistency, security issues, and likely AWS cost factors.
10. **Refine:** Present the final folder structure and full contents of each file, followed by a short checklist of values and settings I must configure.

Do not claim that the code or workflow was tested unless you actually tested it. Clearly label any assumptions and placeholders.

Multi-agent ReAct-style prompt



Act as a multi-agent AWS DevOps team. Help me automate deployment of a small AWS environment using Terraform and GitHub Actions.

The environment should include:

- One VPC

- Two public subnets in different Availability Zones
- An internet gateway and route tables
- One Amazon Linux EC2 instance
- A security group that allows SSH only from an IP address I provide

I am a beginner. Use a Reason–Action–Observation–Refine process. Show concise explanations and results; do not reveal private chain-of-thought.

Assign these roles:

1. **AWS Architect:** Propose the architecture, list assumptions, and identify values I need to provide.
2. **Terraform Engineer:** Design the folder structure and write the complete Terraform configuration using variables and outputs.
3. **Automation and Security Engineer:** Create the GitHub Actions workflow for `terraform fmt`, `terraform validate`, and `terraform plan` on pull requests. Add an approved deployment step for `terraform apply`. Prefer GitHub Actions OIDC for AWS authentication; never hard-code credentials.
4. **Independent Reviewer:** Check the architecture, Terraform references, workflow paths, security settings, and likely cost factors. Identify problems and suggest fixes.
5. **Integrator:** Incorporate the review and present one consistent final project.

Have the agents work on their assigned tasks, then combine their findings before presenting the final answer.

For each stage:

- **Reason:** State the assumptions, requirements, or issue being addressed.
- **Action:** Do the assigned design, coding, or review work.
- **Observation:** Report the result and any problems found.
- **Refine:** Apply needed corrections before moving to the next stage.

The final answer should include:

- A short architecture explanation
- The project folder structure
- The full contents of every Terraform and GitHub Actions file
- Secure AWS and GitHub setup instructions
- Commands and steps to test, verify, and destroy the resources
- A checklist of values and settings I must configure

Use a safe example value if a detail is missing, and clearly mark it for me to replace. Require human approval before applying infrastructure changes. Do not claim that code or workflows were tested unless they were actually tested.